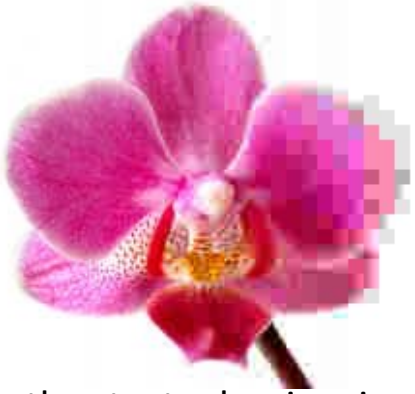
Lossy compression

The JPEG Standard

- *Lossy JPEG* uses *transform coding*. It employs a compression algorithm developed by the Joint Photographic Experts Group JPEG that trades-off detail in the displayed picture for a smaller storage file
- JPEG is the format most used for storing and transmitting photographs on the web. It is preferred to formats such as GIF, which has a limit of 256 distinct colors that is insufficient for colour photographs, and PNG, which produces much larger image files



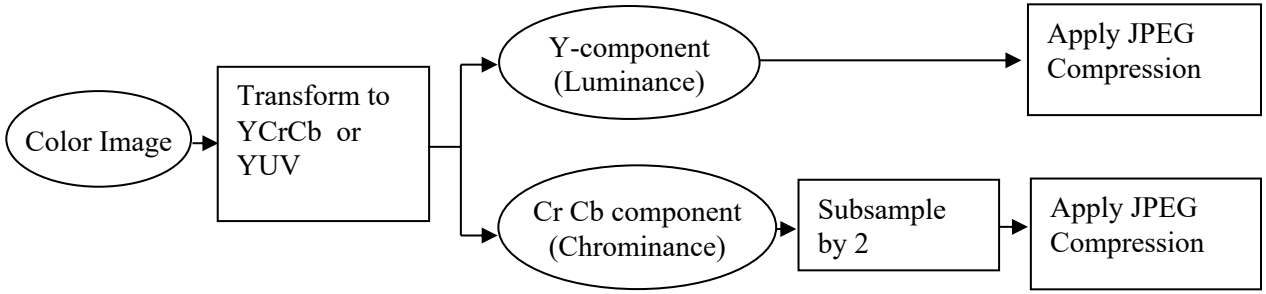
- The compression algorithm is *not* well suited for line drawings and other textual or iconic graphics. The PNG and GIF formats are preferred for these types of images.

- JPEG specifies both the codec defining how an image is transformed into a stream of bytes, and the file format JFIF (JPEG File Intechange Format) used to contain that stream.

JPEG for color images

- With JPEG the image should be converted from RGB into a different color space called YCbCr or Y'CbCr.

- It allows greater perceptual image quality for the same compression because the brightness information, which is more important to the perceptual quality of the image, is confined to a single channel and the choma channel is downsampled.
- JPEG is applied to each channel separately



Chroma subsampling

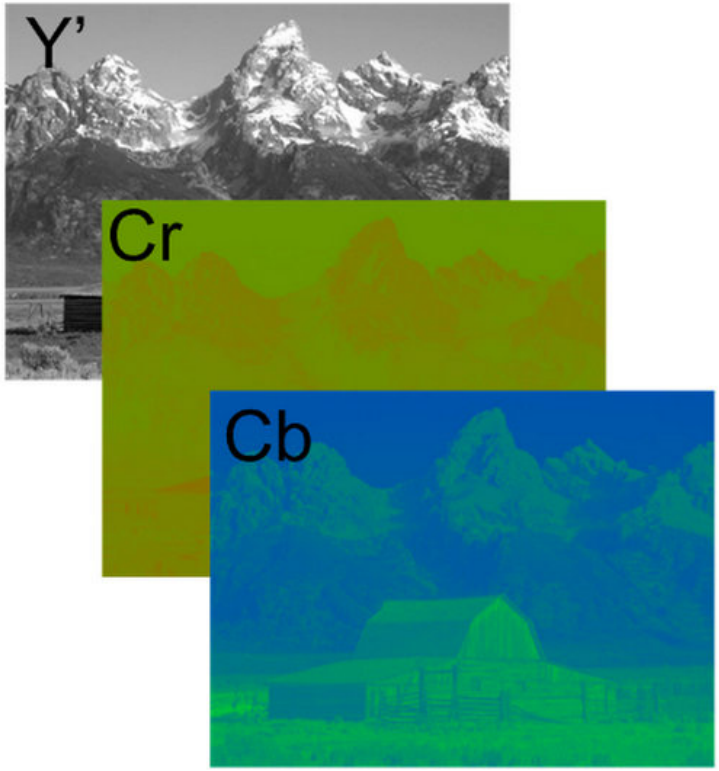
- The different levels of YCbCr subsampling are :
  - 4:4:4 – The resolution of chrominance information is preserved at the same rate as the luminance information. (1 × 1, subsampling disabled)
  - 4:2:2 – Half of the horizontal resolution in the chrominance is dropped, while the full resolution is retained in the vertical direction, with respect to the luminance. (2 × 1 chroma subsampling)
  - 4:1:1 – Only a quarter of the chrominance information is preserved in the horizontal direction with respect to the luminance information
  - 4:2:0 – With respect to the information in the luminance channel, the chrominance resolution in both the horizontal and vertical directions is cut in half (2 × 2 chroma subsampling)

- Photoshop automatically switches to 4:2:0 from 4:4:4 when compression is set at 50% or below.

Y'CbCr color space

- YCbCr and Y'CbCr is a family of color spaces used in video and digital photography systems:
  - Y (Y') is the *luminance* (*luma*) component and is obtained as a combination of R G and B.
  - Cb and Cr are the *blue-difference* and *red-difference* chroma components and are obtained instead subtracting Y from B and R signals respectively

Original image



Y and Y’

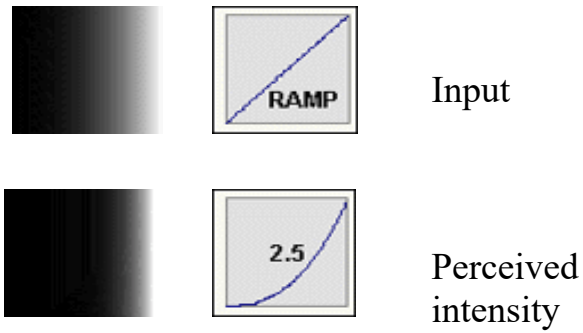
- The Y’ *luma* distinguishes from Y *luminance*, in that light intensity is non-linearly encoded using gamma

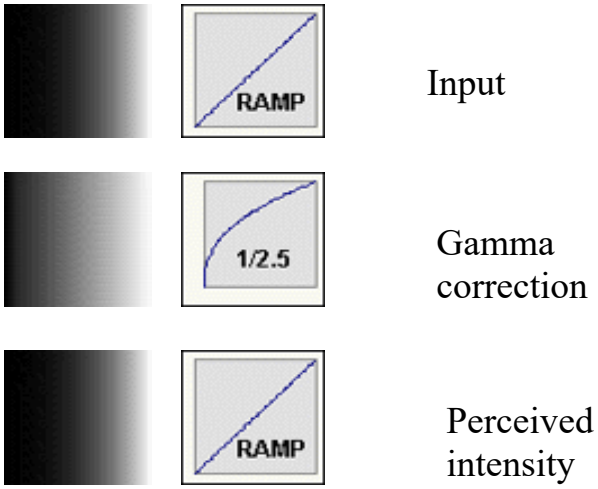
With a digital camera when twice the number of photons hit the sensor it receives twice the signal (a **linear** relationship)

That's not how our eyes work.
We perceive twice the light as being only a fraction brighter and increasingly so for higher light intensities (a **nonlinear** relationship)

Gamma correction is applying an exponential correction so that intensity perceived is exactly twice (re-establish the **linear** relationship)

$$D = (I)^{\gamma}$$

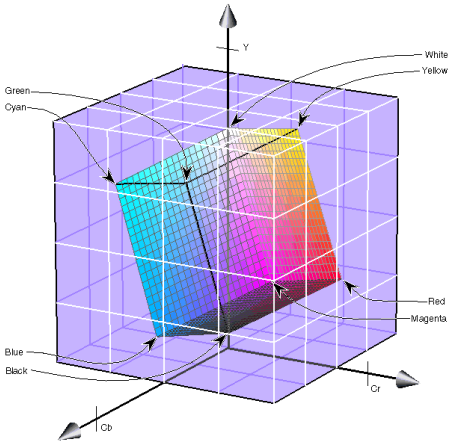




- Colors are distorted passing from RGB to YCbCr or Y'CbCr. The results are scaled and rounded, and offsets are typically added.

- A particular conversion to $Y'C_BC_R$ is specified in the JFIF standard, and should be performed for the resulting JPEG file to have maximum compatibility (256 levels from 8-bit RGB):

$$
\begin{aligned}
Y &= 0.299\,R + 0.587\,G + 0.114\,B \\
Cb &= -0.1687\,R - 0.3313\,G + 0.5\,B + 128 \\
Cr &= 0.5\,R - 0.4187\,G - 0.0813\,B + 128
\end{aligned}
$$



compression factor 10 (39 kb)

compression factor 15 (30 kb)

compression factor 20 (26 KB)

compression factor 30 (16 kb)

compression factor 40 (11 kb)

compression factor 50 (9 KB)

Basic founding principles

- JPEG is based on the following two observations:
  - a large majority of *useful image contents change relatively* slowly across images
  - humans are *more receptive to the loss of higher spatial frequency* components than the loss of lower frequency components

- As a result:
  - It is unusual for intensity values to alter up and down several times in a small area (like f.e. within an 8 x 8 pixel image block)
  - Low spatial frequency components contain more information than high frequency components (that correspond to less useful details and noises)

JPEG major steps

- Consider small image blocks (8x8) to drop out high frequency components

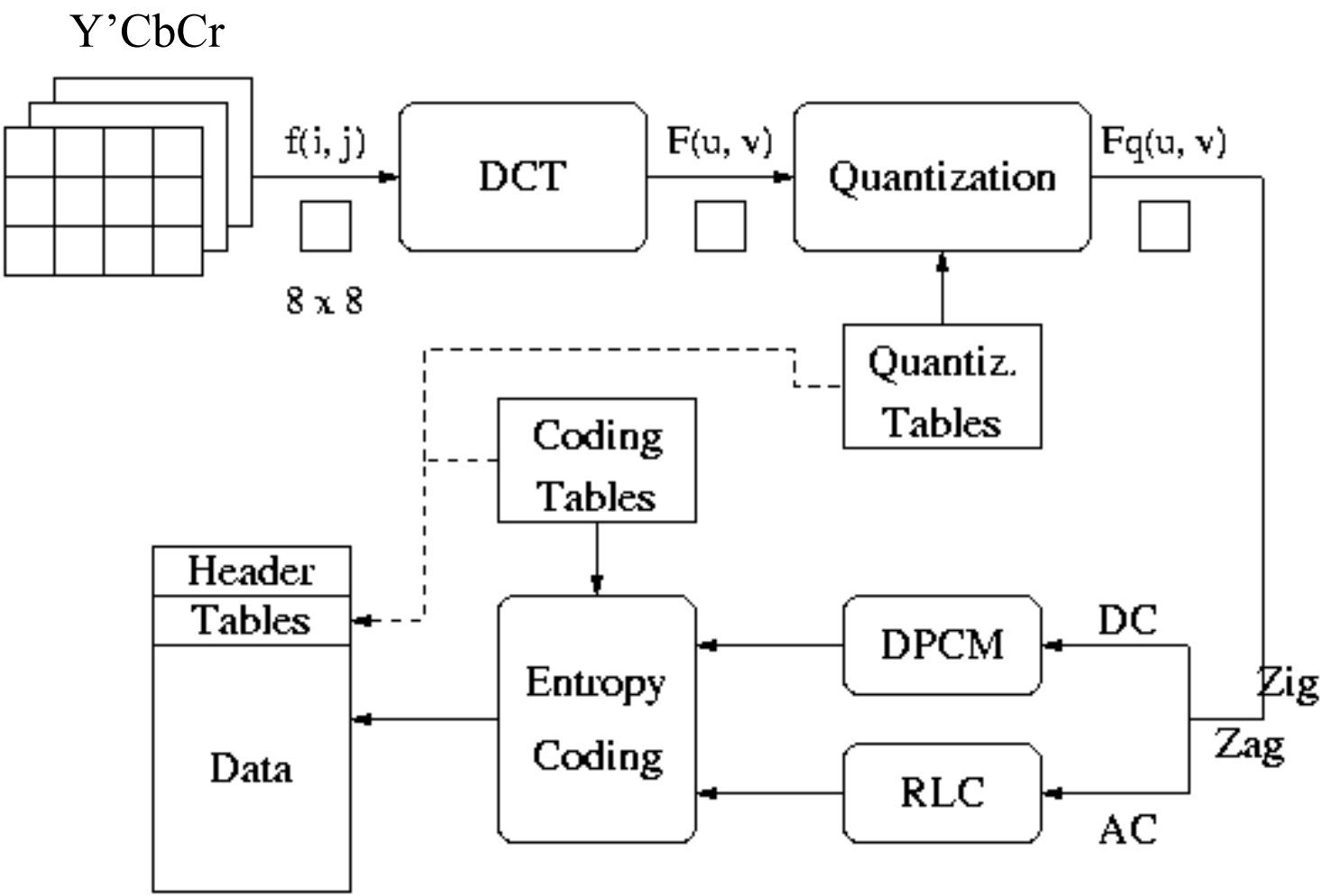
- Apply to (8x8) image blocks :
  - DCT Discrete Cosine Transformation
  - Quantization
  - Zigzag Scan
  - Entropy coding

    Coefficient encoding
    - DPCM on DC component
    - RLE on AC Components

    Huffman Coding

Entropy measures the amount of uncertainty or information present in a random signal which can also be interpreted as the lower limit of data compression

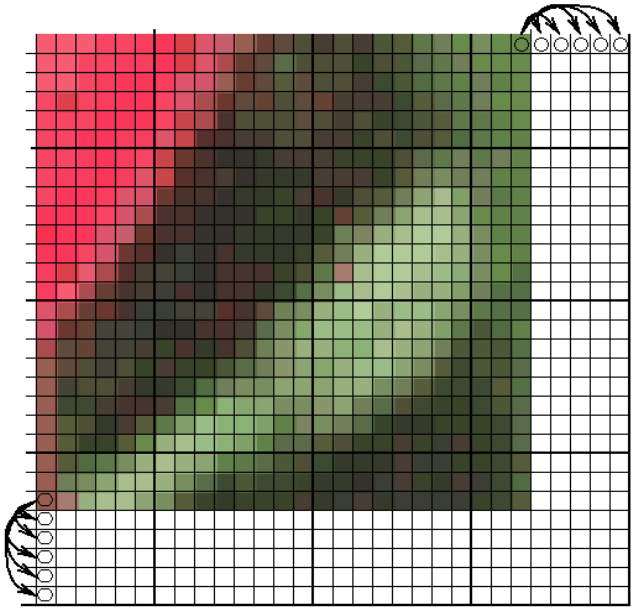
JPEG chain


Y'CbCr
f(i, j)
DCT
F(u, v)
Quantization
Fq(u, v)
8 x 8
Quantiz.
Tables
Coding
Tables
Header
Tables
Data
Entropy
Coding
DPCM
RLC
DC
AC
Zig
Zag

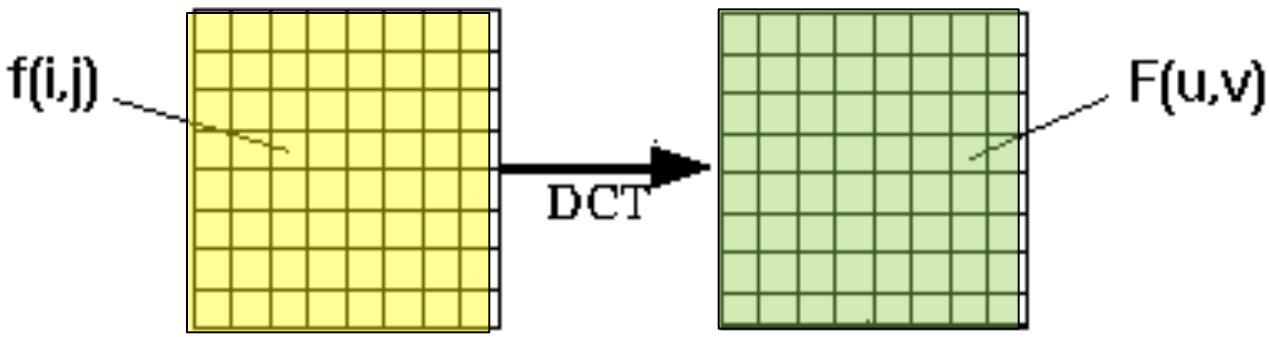
DCT (Discrete Cosine Transformation)

- Apply DCT to 8x8 image blocks

- If the image size is not a multiple of 8, then add copies of the last row or column until a multiple of 8 is reached. This makes both tone and luminance of the 8x8 block not change too much after DCT, as it would be if these elements were set to 0



- DCT allows to shift from spatial domain to frequency domain:

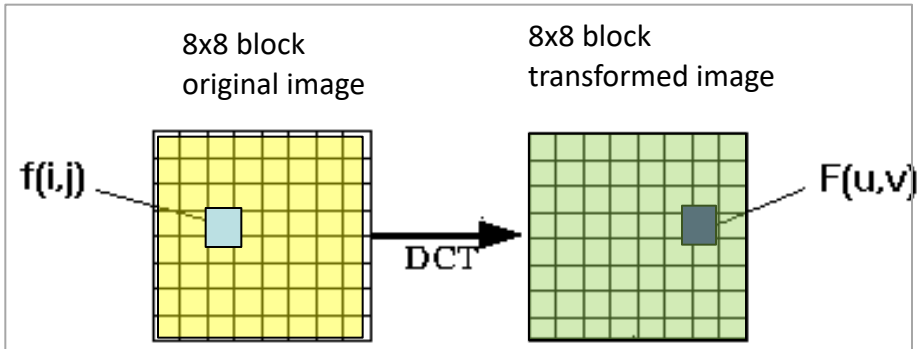


- f(i,j) is the value that is present in the (i,j) position of the 8x8 block of the original image
- F(u,v) is the DCT coefficient in the (u,v) position of the 8x8 matrix that encodes the transformed coefficients

Discrete Cosine Transform (DCT)

$$F(u,v) = \frac{\Lambda(u)\Lambda(v)}{4}\sum_{i=0}^{7}\sum_{j=0}^{7}\cos\frac{(2i+1)\cdot u\pi}{16}\cdot\cos\frac{(2j+1)\cdot v\pi}{16}\cdot f(i,j)$$

$$\Lambda(\xi) = \begin{cases} \frac{1}{\sqrt{2}} & for\ \xi = 0 \\ 1 & otherwise \end{cases}$$



f(i,j) is the value that is present in the (i,j) position of the 8x8 block of the original image

F(u,v) is the DCT coefficient in the (u,v) position of the 8x8 matrix that encodes the transformed coefficients

Inverse Discrete Cosine Transform (IDCT)

$$\hat{f}(i,j) = \frac{1}{4}\sum_{u=0}^{7}\sum_{v=0}^{7}\Lambda(u)\Lambda(v)\cos\frac{(2i+1)\cdot u\pi}{16}\cdot\cos\frac{(2j+1)\cdot v\pi}{16}\cdot F(u,v)$$

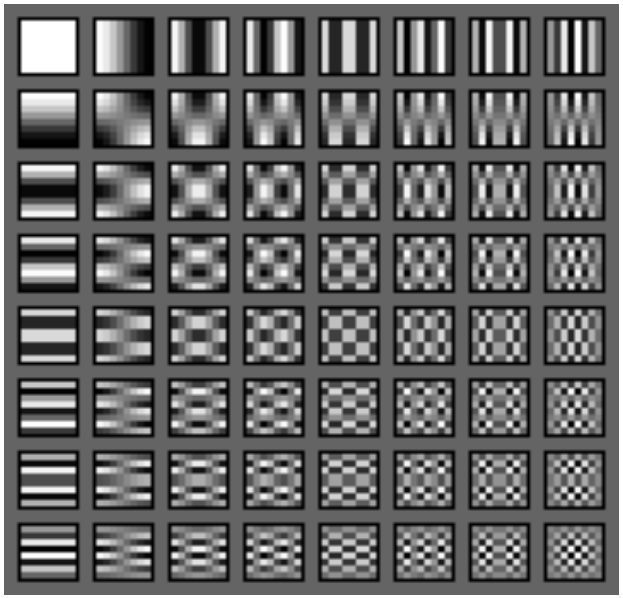
$$\Lambda(\xi) = \begin{cases} \frac{1}{\sqrt{2}} & for\ \xi = 0 \\ 1 & otherwise \end{cases}$$

Visualizing DCT

The 64 (8 x 8) DCT basis functions

Frequency along row
(vertical edges)

F[0,0]



Frequency along column
(horizontal edges)

DCT Example

Input Pixel Matrix

140	144	147	140	140	155	179	175
144	152	140	147	140	148	167	179
152	155	136	167	163	162	152	172
168	145	156	160	152	155	136	160
162	148	156	148	140	136	147	162
147	167	140	155	155	140	136	162
136	156	123	167	162	144	140	147
148	155	136	155	152	147	147	136

Output DCT Matrix

larger numbers

186	-18	15	-9	23	-9	-14	19
21	-34	26	-9	-11	11	14	7
-10	-24	-2	6	-18	3	-20	-1
-8	-5	14	-15	-8	-3	-3	8
-3	10	8	1	-11	18	18	15
4	-2	-18	8	8	-4	1	-7
9	1	-3	4	-1	-7	-1	-2
0	-8	-2	2	1	4	-6	0

Smaller numbers

Quantization

- To reduce number of bits per sample, quantization is used. It is dividing the value with a constant and rounding to the integer:

  $$F'[u, v] = \text{round}\,(F[u, v] / q[u, v])$$

  where q(u,v) is the quantization matrix and F(u,v) is the DCT coefficient matrix.

  Example: 101101 = 45 (6 bits)

  q[u, v] = 4 ⇒ F[u, v] / q[u, v] = 11 = 1011

- Quantization error is the main source of the lossy compression. Different quantization matrices can be used.

- *Uniform Quantization*
  Each F[u,v] is divided by the same constant *N*.

- *Non-uniform Quantization*
  accounts for the fact that human eye is most sensitive to low frequencies (upper left corner), less sensitive to high frequencies (lower right corner), more sensitive to luminance, less to color

Luminance Quantization Table q(u, v)

16	11	10	16	24	40	51	61
12	12	14	19	26	58	60	55
14	13	16	24	40	57	69	56
14	17	22	29	51	87	80	62
18	22	37	56	68	109	103	77
24	35	55	64	81	104	113	92
49	64	78	87	103	121	120	101
72	92	95	98	112	100	103	99

Chrominance Quantization Table q(u, v)

17	18	24	47	99	99	99	99
18	21	26	66	99	99	99	99
24	26	56	99	99	99	99	99
47	66	99	99	99	99	99	99
99	99	99	99	99	99	99	99
99	99	99	99	99	99	99	99
99	99	99	99	99	99	99	99
99	99	99	99	99	99	99	99

139	144	149	153	155	155	155	155
144	151	153	156	159	156	156	156
150	155	160	163	158	156	156	156
159	161	162	160	160	159	159	159
159	160	161	162	162	155	155	155
161	161	161	161	160	157	157	157
162	162	161	163	162	157	157	157
162	162	161	161	163	158	158	158

(a) source image samples

235.6	-1.0	-12.1	-5.2	2.1	-1.7	-2.7	1.3
-22.6	-17.5	-6.2	-3.2	-2.9	-0.1	0.4	-1.2
-10.9	-9.3	-1.6	1.5	0.2	-0.9	-0.6	-0.1
-7.1	-1.9	0.2	1.5	0.9	-0.1	0.0	0.3
-0.6	-0.8	1.5	1.6	-0.1	-0.7	0.6	1.3
1.8	-0.2	1.6	-0.3	-0.8	1.5	1.0	-1.0
-1.3	-0.4	-0.3	-1.5	-0.5	1.7	1.1	-0.8
-2.6	1.6	-3.8	-1.8	1.9	1.2	-0.6	-0.4

(b) forward DCT coefficients

16	11	10	16	24	40	51	61
12	12	14	19	26	58	60	55
14	13	16	24	40	57	69	56
14	17	22	29	51	87	80	62
18	22	37	56	68	109	103	77
24	35	55	64	81	104	113	92
49	64	78	87	103	121	120	101
72	92	95	98	112	100	103	99

(c) quantization table

16	11	10	16	24	40	51	61
12	12	14	19	26	58	60	55
14	13	16	24	40	57	69	56
14	17	22	29	51	87	80	62
18	22	37	56	68	109	103	77
24	35	55	64	81	104	113	92
49	64	78	87	103	121	120	101
72	92	95	98	112	100	103	99

(c) quantization table

235.6	−1.0	−12.1	−5.2	2.1	−1.7	−2.7	1.3
−22.6	−17.5	−6.2	−3.2	−2.9	−0.1	0.4	−1.2
−10.9	−9.3	−1.6	1.5	0.2	−0.9	−0.6	−0.1
−7.1	−1.9	0.2	1.5	0.9	−0.1	0.0	0.3
−0.6	−0.8	1.5	1.6	−0.1	−0.7	0.6	1.3
1.8	−0.2	1.6	−0.3	−0.8	1.5	1.0	−1.0
−1.3	−0.4	−0.3	−1.5	−0.5	1.7	1.1	−0.8
−2.6	1.6	−3.8	−1.8	1.9	1.2	−0.6	−0.4

Coefficienti DCT

$F(u,v)$

Quantizzazione (fase di codifica)

15	0	−1	0	0	0	0	0
−2	−1	0	0	0	0	0	0
−1	−1	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

Coefficienti quantizzati

$$F^{Q}(u,v) = \text{round int}\left(\frac{F(u,v)}{Q(u,v)}\right)$$

Dequantizzazione (fase di decodifica)

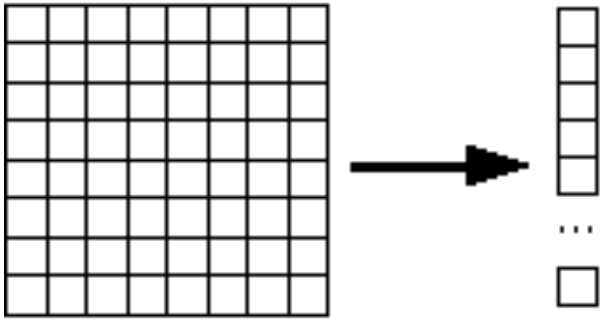
240	0	−10	0	0	0	0	0
−24	−12	0	0	0	0	0	0
−14	−13	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

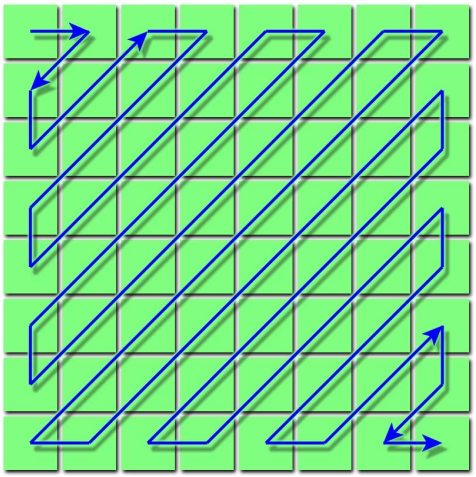
Coefficienti dequantizzati

$$F^{Q'}(u,v) = F^{Q}(u,v) \cdot Q(u,v)$$

Zig-zag scan

- As a result of quantization we have a 8x8 matrix with many elements equal to 0. Non null coefficients are all in the upper left corner.
- This suggests to transform the 8x8 matrix into a 64 element vector using a zig-zag order. Zig-Zag scan is used to group low frequency coefficients in the top of the vector: *maps 8x8 to 1x64 vector*





Coefficient encoding

- Coefficients are encoded differently:
  - Differential Pulse Code Modulation (DPCM) on the DC component
    - DC component is large and varied, but often close to the value of the DC component of the previous block
    - According to this JPEG encodes the difference between the previous and the current 8 x 8 block (DC diff)
  - Run Length Encoding (RLE) on AC components
    - Many of the AC coefficients are equal to 0
    - According to this they are encoded using RLE, which *counts the number of consecutive 0s* A minimum of 0 to a maximum of 16 consecutive 0s is allowed. In the latter case the special symbol (15,0) is used.
    - The end of block is encoded with (0,0). Can be used to end the sequence prematurely if all the coefficients are 0.

– For the DC component (DC diff) we build the pair: (**SIZE**) (**AMPLITUDE**) where:

SIZE is the number of bits needed to represent the DC difference value;
AMPLITUDE is the value of the DC difference .

SIZE	Value
1	-1, 1
2	-3, -2, 2, 3
3	-7..-4, 4..7
4	-15..-8, 8..15
.	.
.	.
10	-1023..-512, 512..1023

Example: if DC value is 4, 3 bits are needed.

– For the AC components the representation (**RUNLENGHT**, **SIZE**) (**AMPLITUDE**) is used where:

RUNLENGHT is the number of consecutive 0 (from 0 to 15),
SIZE has the same meaning as for the DC coefficient
AMPLITUDE is the actual value for nonzero AC coefficients

DC Coefficient

63 Coefficienti AC

15	0	-1	0	0	0	0	0
-2	-1	0	0	0	0	0	0
-1	-1	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

DC of preceding block = 12, 15 – 12 = +3

3, (1 zero), –2, –1, –1, –1, (2 zero), -1 , (55 zero)

	symbol-1	symbol-2
DC Coeff.	(size)	(amplitude)
AC Coeff.	(runlength,size)	(amplitude)
Special symbols (symbol-1)		
(15,0)	(15,0) – runlength=16	
(0,0)	(0,0) – EOB (end of block)	

Size	Amplitude	
1	-1	1
2	-3,-2	2,3
3	-7...-4	4..7
4	-15..-8	8..15
5	-31..-16	16..31
6	-63..-32	32..63
7	-127..-64	64..127
8	-255..-128	128..255
9	-511..-256	256..511
10	-1023..-512	512..1023

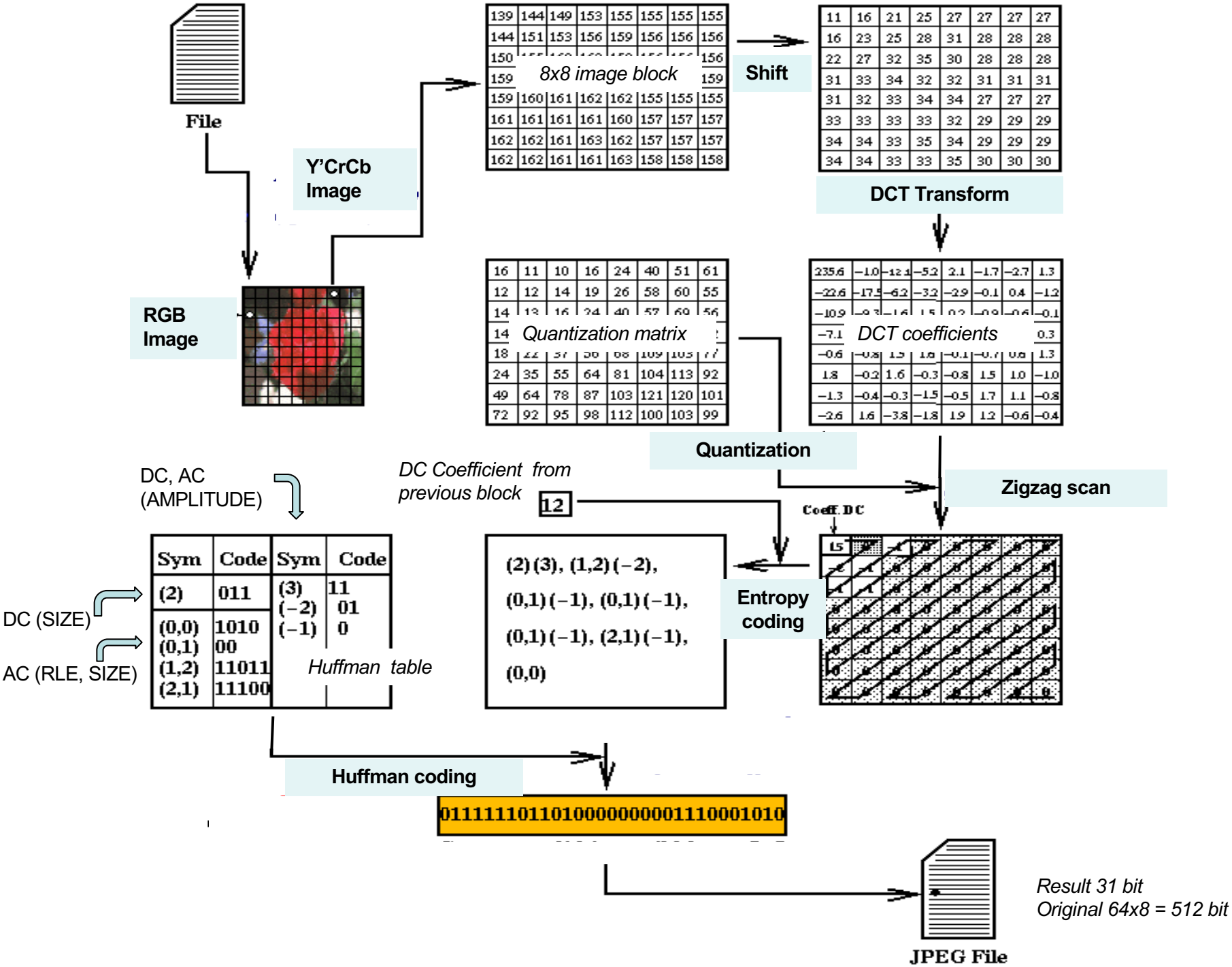
(2)(3), (1,2)(–2), (0,1)(–1), (0,1)(–1), (0,1)(–1), (2,1)(–1), (0,0)

Baseline Entropy Coding

Huffman encoding

- After we have encoded every block, we have a sequence of symbols:
  - Symbol 1: (SIZE) or (RLE, SIZE)
  - Symbol 2: (AMPLITUDE)
- These symbols are further encoded using the Huffman encoding to reduce the number of data.

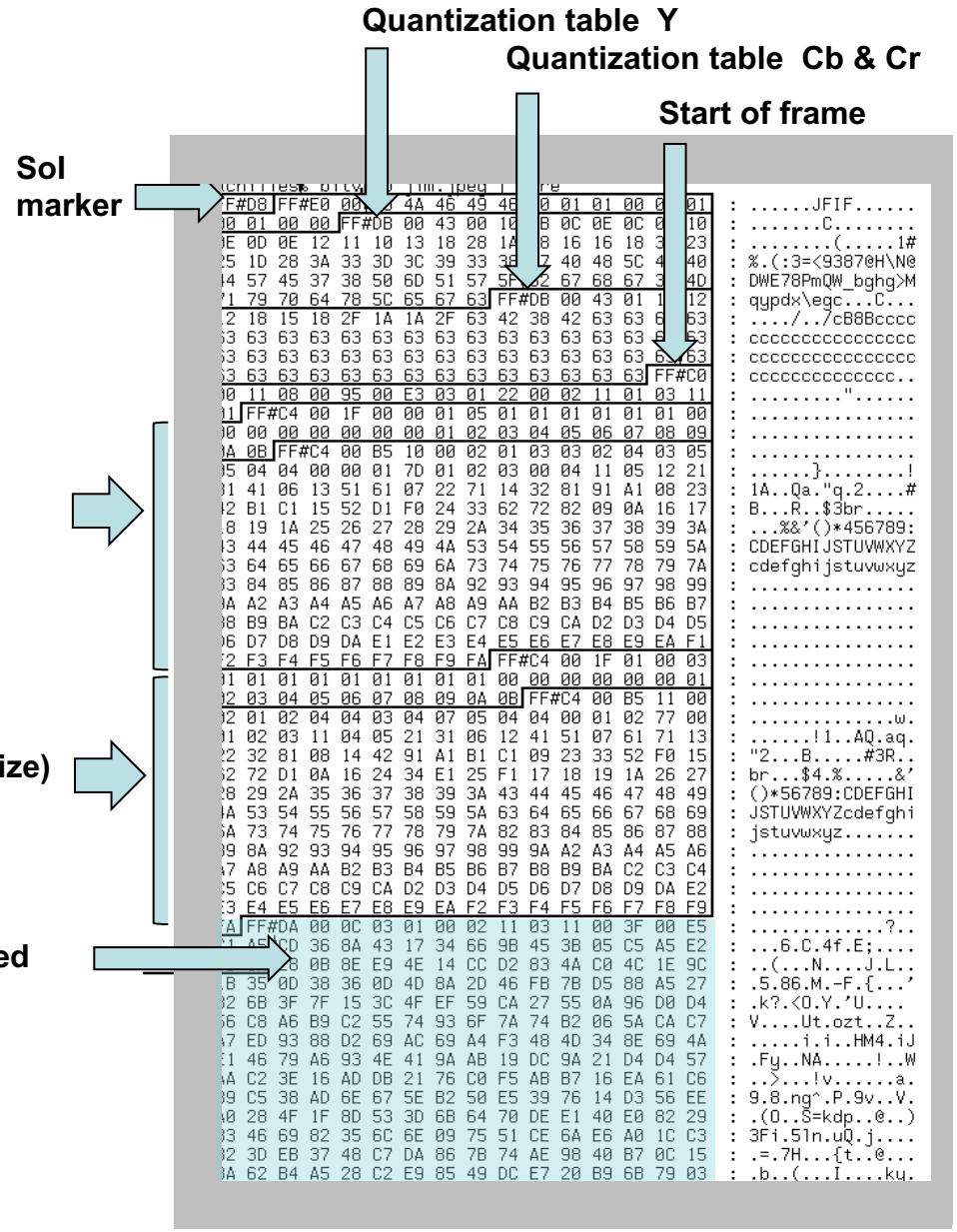
- Most frequent symbols are encoded with shorter codes. Less frequent with longer ones. Huffman tables provide codes for every symbol of the sequence. Huffman tables are different for DC and AC symbol 1.
- Huffman Tables can be custom (sent in header) or default.


File
Y'CrCb Image
RGB Image
8x8 image block
Shift
DCT Transform
Quantization matrix
DCT coefficients
Quantization
Zigzag scan
Coeff. DC
DC, AC (AMPLITUDE)
DC Coefficient from previous block
12
DC (SIZE)
AC (RLE, SIZE)
Sym Code Sym Code
(2) 011 (3) 11
(−2) 01
(−1) 0
(0,0) 1010
(0,1) 00
(1,2) 11011
(2,1) 11100
Huffman table
(2)(3), (1,2)(−2),
(0,1)(−1), (0,1)(−1),
(0,1)(−1), (2,1)(−1),
(0,0)
Entropy coding
Huffman coding
01111110110100000000011100010 10
JPEG File
Result 31 bit
Original 64x8 = 512 bit

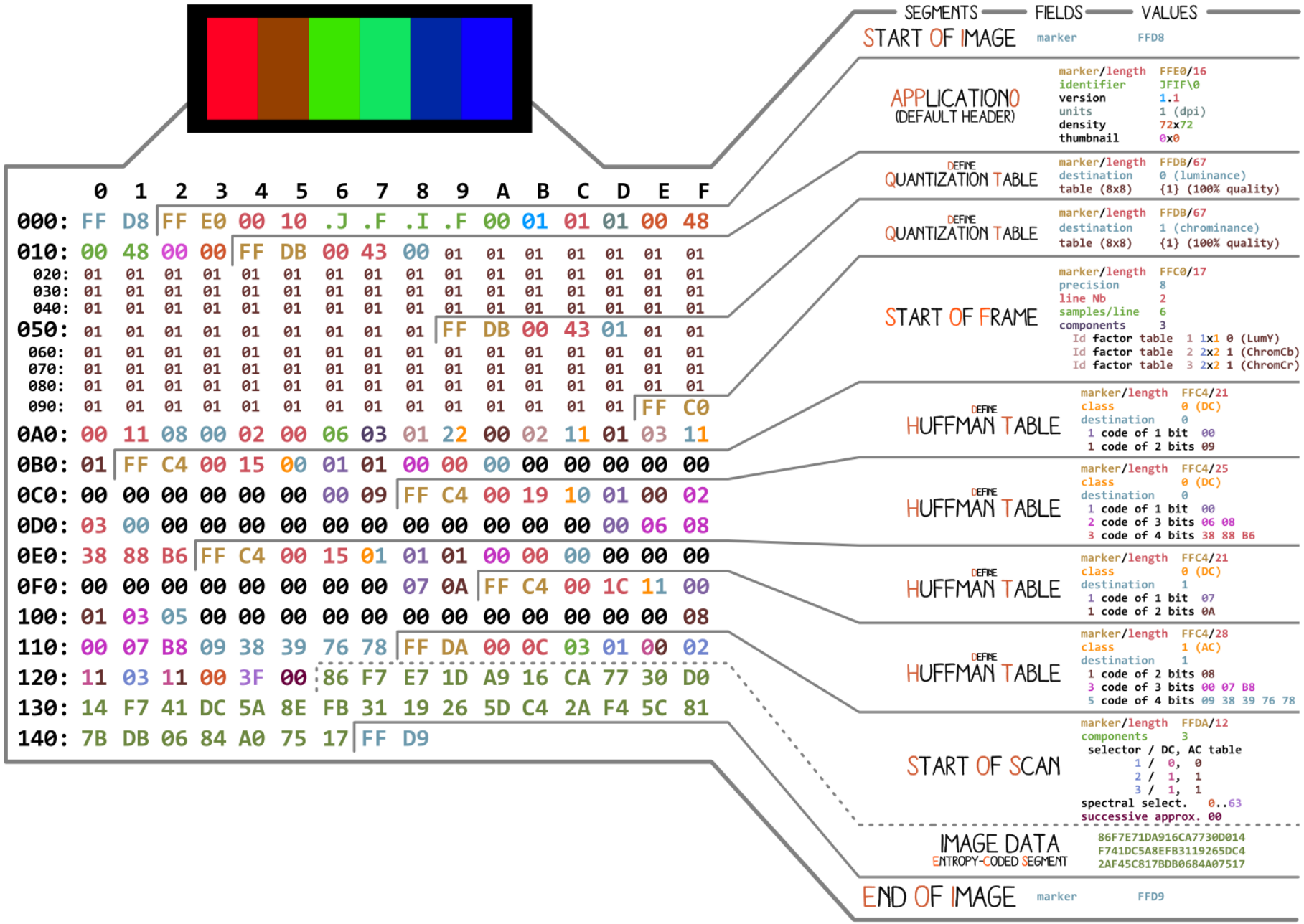
The JPEG Bitstream

JPEG image format

Type of segmen	Length of segment
Start of image	0
APP0	16
Quantisation table	67
Quantisation table	67
Start of frame: baseline DCT	11
Huffman table	28
Huffman table	63
Huffman table	28
Huffman table	63
Start of scan	49363
End of image	0



The JPEG Bitstream



GIF, PNG, TIFF, JPEG standards at comparison

GIF – PNG

- Both GIF and PNG formats use lossless compression to achieve medium levels of compression on images
  - GIF works best on images with few colors or images in which one color is dominant. It acts best on identical, adjacent pixels (or rows of identical, adjacent pixels).
  - There is no loss during the compression process as long as the original image had fewer than 256 colors. The key to using GIFs effectively is to use the smallest possible number of colors. Also notable about GIFs is the fact that images can be transparent, animated, or both.
  - PNG is the other choice with higher number of colors.

JPEG

- The JPEG format uses lossy compression to achieve high levels of compression on images with many colors.

- The compression works best with continuous-tone images, that is, images where the change between adjacent pixels is small but not zero. JPEG images generally store 16 or 24 bits of color and thus are best for 16- or 24-bit images.
- Due to the noticeable loss of quality during the compression process, JPEGs should be used only where image file size is important, primarily on web pages.

- JPEG is a standard for digital photographs because it can save information on more than 16 million different hues. Its "lossy" compression has little effect on photographs. JPEG will produce a smaller file than PNG for photographic images (often 5–10 times) with negligible loss in quality.

– PNG is a better choice than JPEG for storing images that contain text, line art, or other images with sharp transitions that do not transform well into the frequency domain.

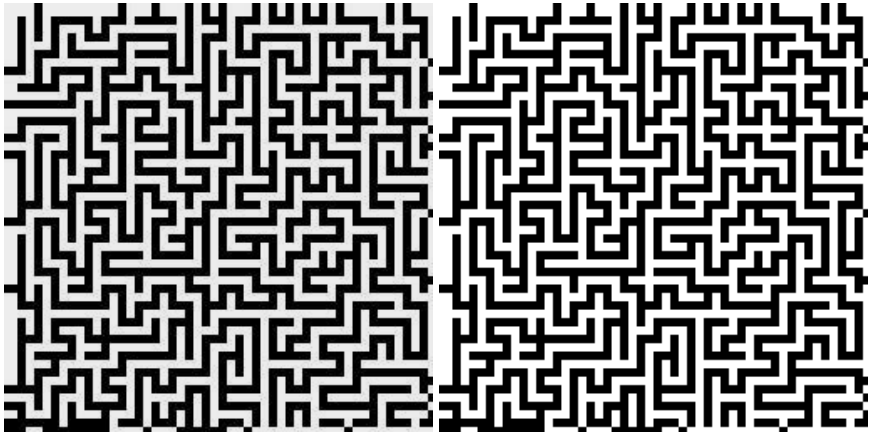
- Where an image contains both sharp transitions and photographic parts a choice must be made between the large but sharp PNG and a small JPEG with artifacts around sharp transitions.

Some general rules

- If your image is...
  - Black and white — Use GIF
  - Text on a plain background — Use GIF
  - Transparent or animated — Use GIF

50%

Black and white



JPG: 26K | GIF: 3.7K

JPG: 2.4K | GIF: 620 bytes

50%

Text on a plain background

This is 32-point, antialiased text.

JPG: 6K

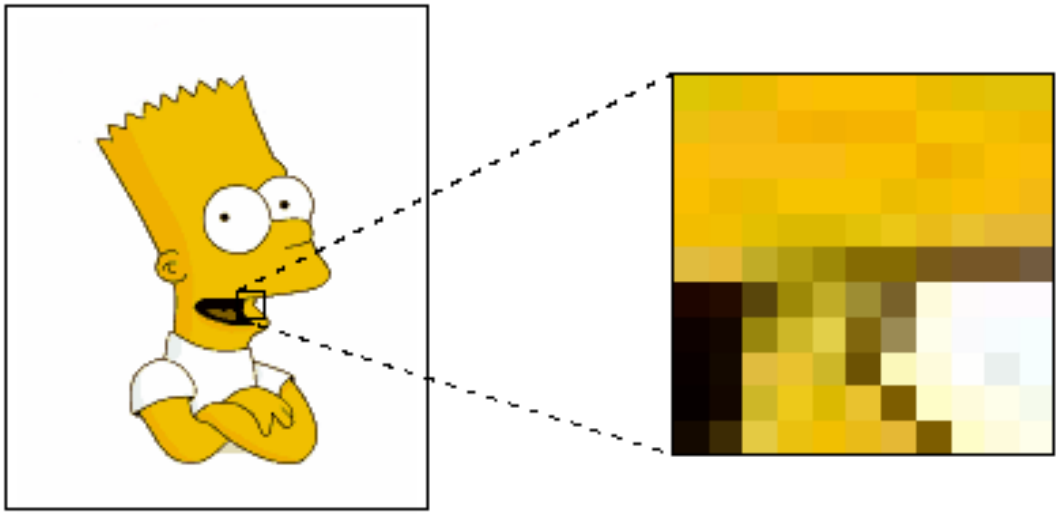
This is 32-point, antialiased text.

GIF: 0.7K

– Computer-drawn line, cartoon art	Use GIF
– Small images, like icons, buttons	Use GIF
– Predominantly (>80%) one color	Consider GIF

Image size: 146 x 184, 75 colors.
File size:

- 24 bpp: 80592 byts
- GIF (8 bpp): 3585 byte (FC=22.48)
- JPG (24 bpp): 4805 byte (FC=16.77)
- PPM.ZIP (24 bpp): 3698 byte (FC=21.79)



- If your image is...
  - 16/24-bit scanned photograph — Use JPEG
  - Computer-drawn continuous-tone art — Use JPEG
  - Scanned images and photographs — Use JPEG
  - (Large) images with a lot of detail — Use JPEG

50%

16- or 24-bit scanned photograph

JPG: 50K	GIF: 113K

50%

Computer-drawn continuous-tone art

JPG: 7K	GIF: 19K

Image size: 244 x 334, 31322 colors
File size:

- 24 bpp: 244488 byte
- GIF (8 bpp): 49613 byte (FC=4.92)
- JPG (24 bpp): 16352 byte (FC=14.95)
- PPM.ZIP (24 bpp): 190977 byte (FC=1.28)

JPEG 2000 Standard

- JPEG 2000 is an ISO Standard (ISO/IEC 15444-1:2000) for images created by the Joint Photograph Expert Group committee in year 2000. Allows multi spectral imaging. JPEG2000 supports both lossy and lossless compression.

- JPEG2000 is not so widely used nor on the web as JPEG. It has no accepted way to embed EXIF data....

- JPEG2000 image files have the extension .jp2 or .j2f

Why another standard?

- An important feature of systems based on JPEG 2000 is the ability to extract a variety of resolutions, components, areas of interest, and compression ratios from a single JPEG 2000 code stream

- JPEG2000 is based on Discrete Wavelet Transformation which creates so-called resolutions, therefore the simplest possibility for bitrate control is discarding resolutions

- The idea is that only several of the smallest resolution levels get transmitted to the client. This option provides a fast way of progressive quality improvement with no special demands/settings during image encoding. Quality layers allow progressive video/image quality improvement

Principal motivations against JPEG

- Low bit-rate compression: at low bit-rates (e.g. below 0.25 bpp for highly detailed gray-level images) the distortion in JPEG becomes unacceptable
- Large images: JPEG doesn't compress images greater than 64x64 without tiling
- Transmission in noisy environments: in JPEG quality suffers dramatically, when bit errors are encountered
- Computer generated imaginary: JPEG is optimized for natural images and performs badly on computer generated images
- Compound documents: JPEG fails to compress bi-level (text) imagery

The JPEG 2000 processing chain

- JPEG2000 properties and encoding/decoding mechanisms are used to achieve constant video bitrate (and thus fulfill various QoS goals), to get most of the connections with very limited bandwidth (e.g. drones) or simply to limit data transmitted while viewing still images

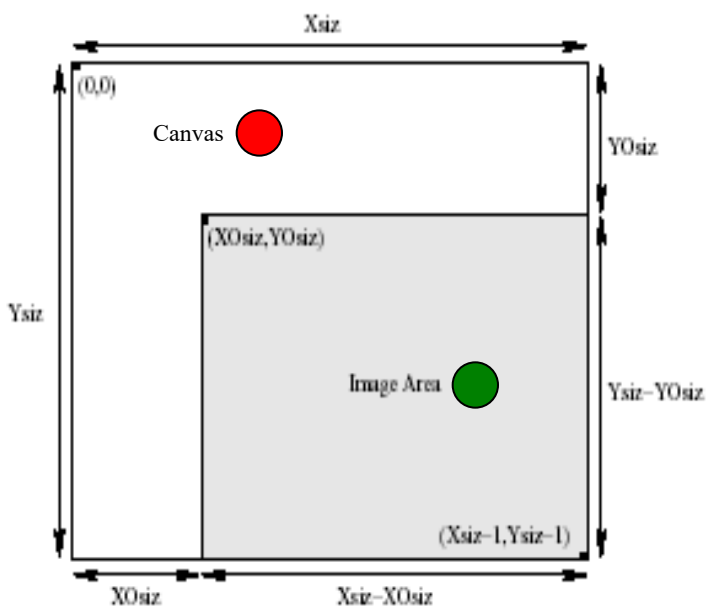
- Similarly to JPEG it requires that the image is preprocessed and converted from RGB into YCbCr or Y'CbCr to allow greater perceptual image quality for the same compression

Reference Grid

- JPEG2000 supports multiple *image components* 1 to 255 or more. Each component can have different sizes and bit depths (1 to 32bits) and different alignments relative to each other This makes it possible to store different parts of the same picture using different quality.

- JPEG2000 uses the concept of the *canvas* to align *multiple image components* in a single coordinate system.

- By default, image components are placed on the canvas so that the *image area* and *canvas* align. Only those samples which fall within the image area actually belong to the image component.



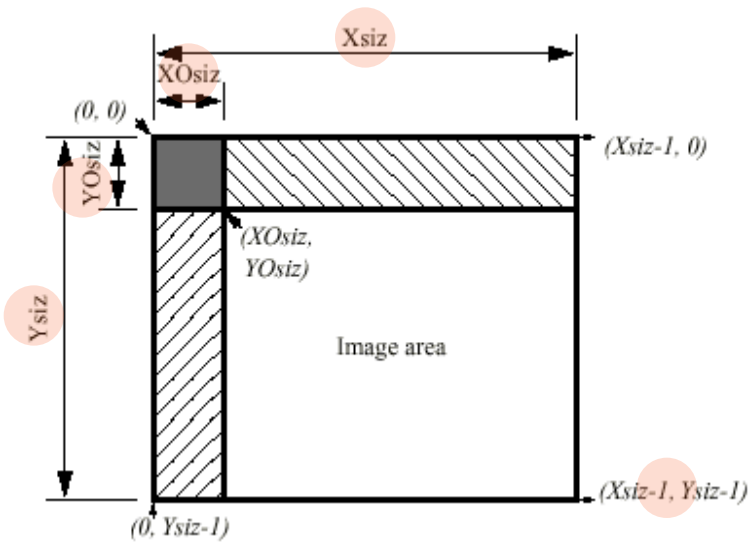
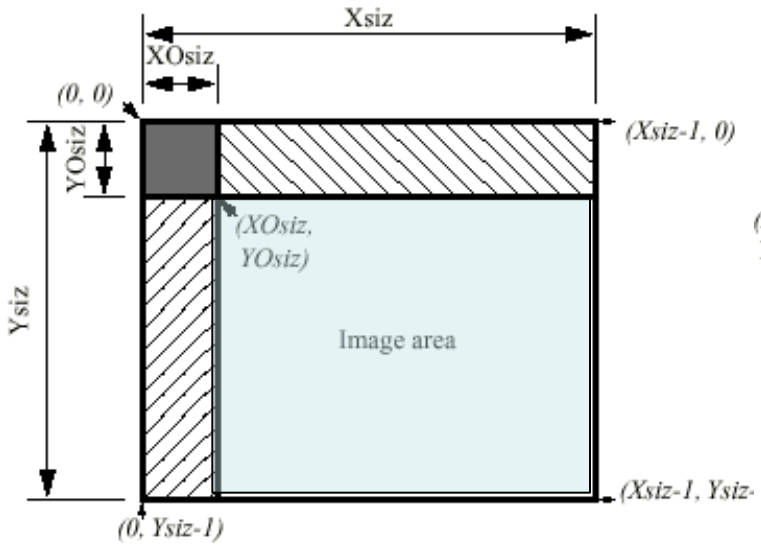
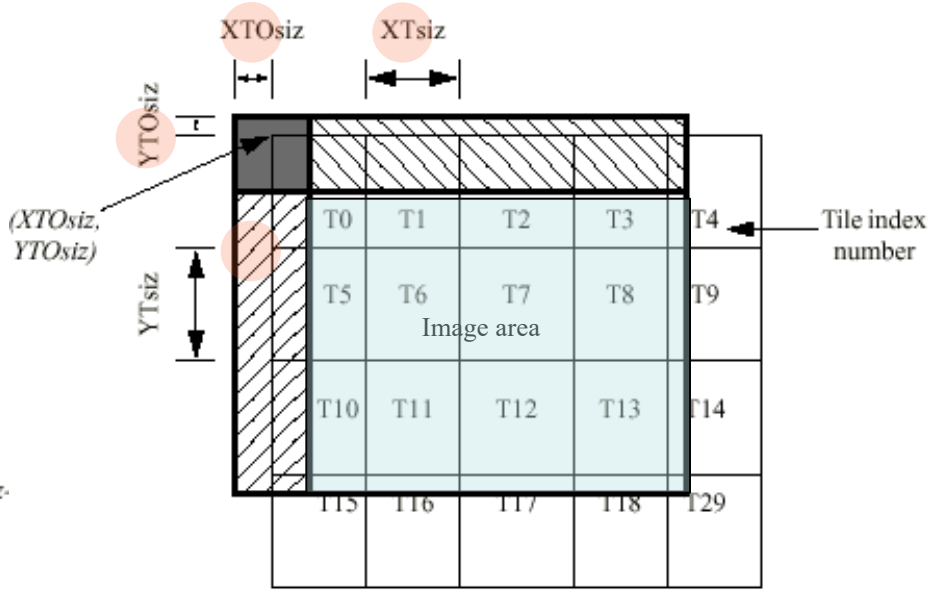


Image Tiling

- Each image component is further broken down into tiles. Tile sizes are variable, and can differ from component to component. Similar to blocks in JPEG, but more flexible.

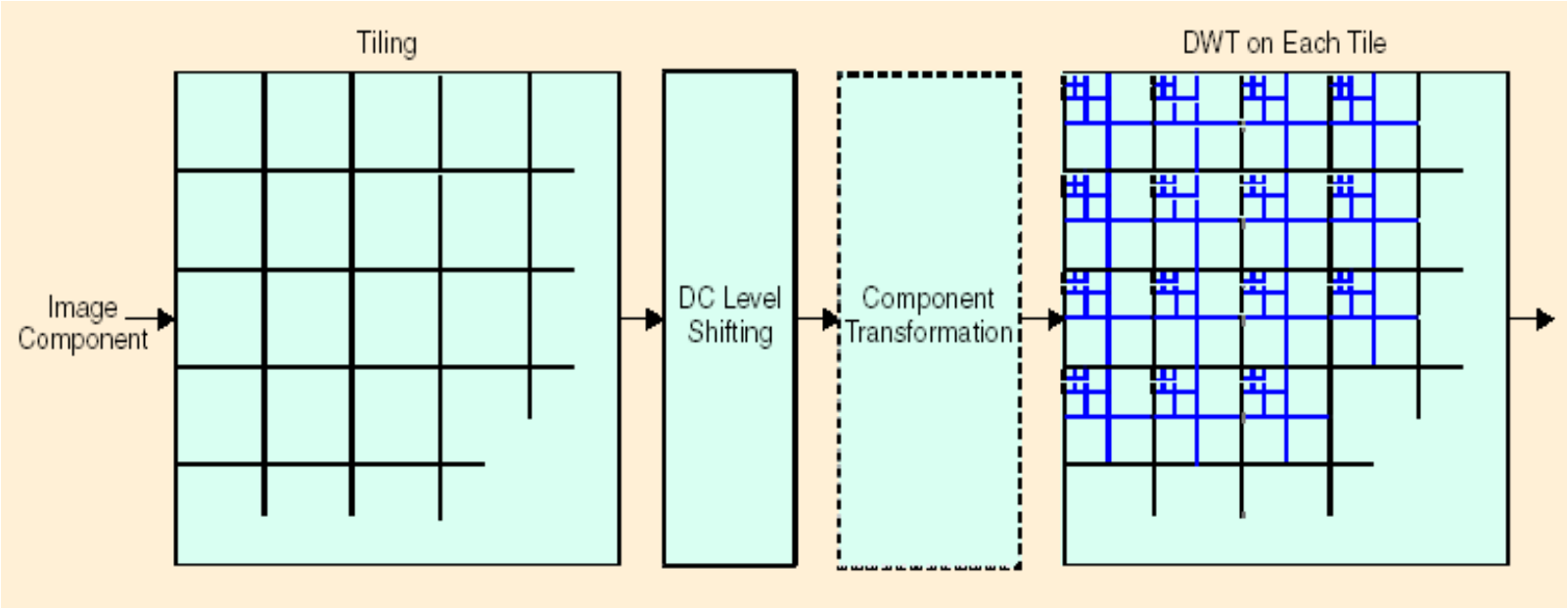
- Image tiles relative to a reference grid. The reference grid is partitioned into a regular sized rectangular array of tiles. The *tile size* and *tiling offset* are defined on the reference grid, by dimensional pairs (XTsiz, YTsiz) and (XTOsiz, YTOsiz), respectively.





Wavelet transform

- JPEG2000 uses Discrete Wavelet Transform in the lossy stage of image compression. Wavelet transform breaks down the image into multi resolution representations.
- A different number of wavelet decomposition levels can be specified for each component. The number of wavelet decomposition levels is also the number of resolution levels available.
- The wavelet transform is applied to the image on a tile by tile basis.



Discrete Wavelet Transform

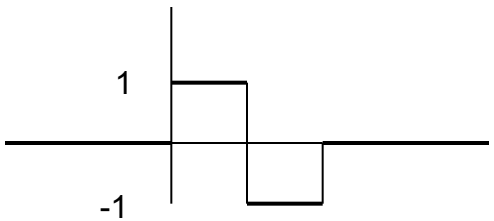
- The first Discrete Wavelet Transform was invented by the mathematician Alfréd Haar:
  - For a list of $2n$ input numbers, the *Haar wavelet* transform simply pairs up input values, ***storing the difference*** and ***passing the sum*.**
  - This ***process is repeated recursively***, pairing up the sums to provide the next scale: finally resulting in $2n - 1$ differences and one final sum.

- The Discrete Wavelet Transform has nice properties that make it an alternative to FFT:
  - it can be performed in $O(n)$ operations
  - it captures not only some notion of the *frequency content* of the input (by examining it at different scales) but also captures *the spatial* (*temporal*) *content* i.e. the location (location in time) at which the frequencies occur

1D Discrete Wavelet Transform

- The Haar wavelet can be described as a step function:

$$F(x) \quad \begin{array}{ll} 1 & 0 <= x < ½ \\ -1 & ½ < \ x <=1 \\ 0 & \text{otherwise} \end{array}$$



2x2 matrix $H = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$

  - Given a sequence ($\mathbf{a_0, a_1, a_2, a_3 \ldots a_{2n+1}}$) of even lenght this can be transformed into a sequence of two-component vectors (($\mathbf{a_0, a_1}$),… ($\mathbf{a_{2n}, a_{2n+1}}$)).

  - If one multiplies each vector with the matrix H one gets the result (($\mathbf{s_0, d_0}$).....($\mathbf{s_n, d_n}$) of one stage of the Haar wavelet transform (***sum, difference***).

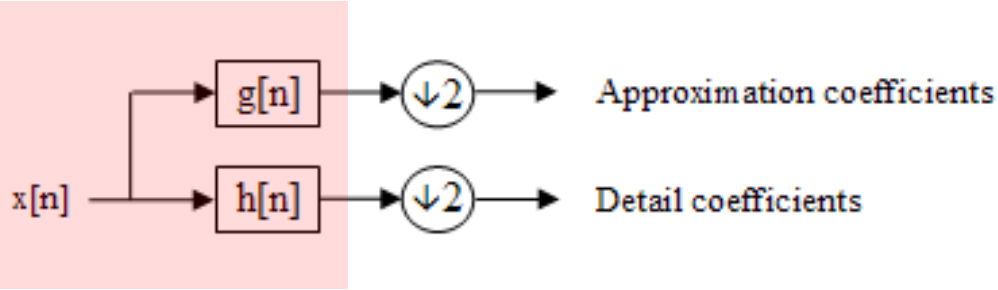
  - The two sequences *s* and *d* are separated and the process is repeated with the sequence ($\mathbf{s_0, s_1, s_2, s_3 \ldots s_n}$)

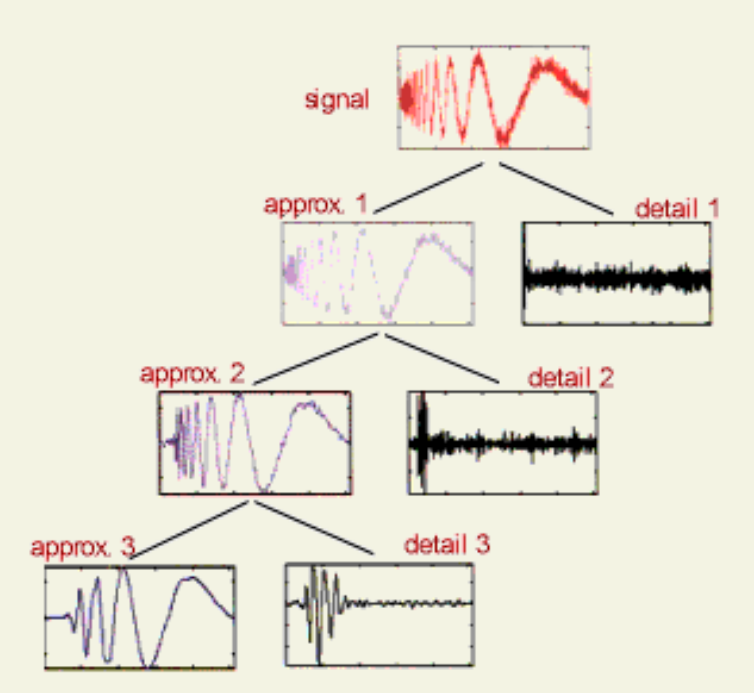
Visualizing the continuous version

- In the one dimensional continuous case the wavelet transform breaks the signal into subbands by passing it through a *low pass filter* and a *high pass filter*. The outputs give:
  - the approximation coefficients (from the *low-pass* filter)
  - the detail coefficients (from the *high-pass* filter)

$$y_{\text{low}}[n] = \sum_{k=-\infty}^{\infty} x[k]g[2n-k]$$

$$y_{\text{high}}[n] = \sum_{k=-\infty}^{\infty} x[k]h[2n-k]$$

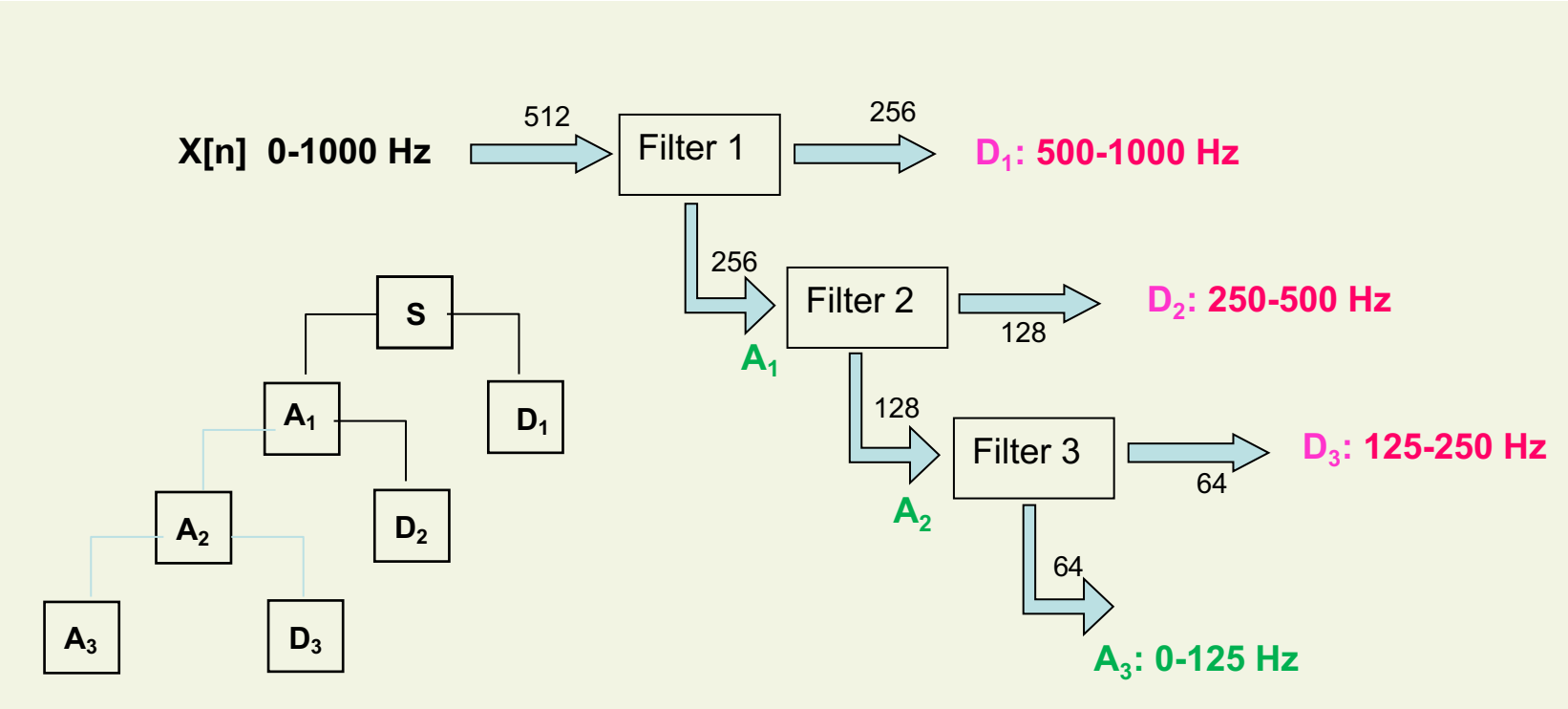




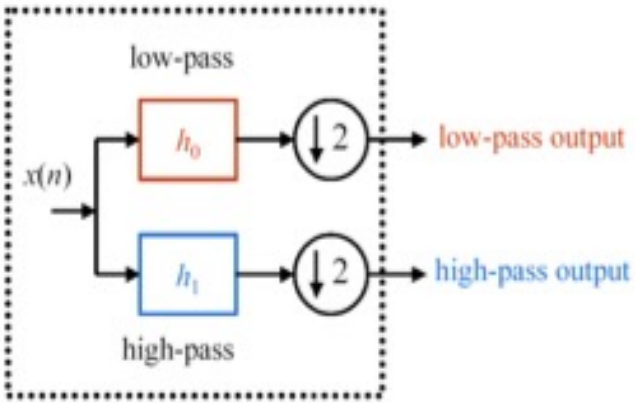
Low pass filter and high pass filtering

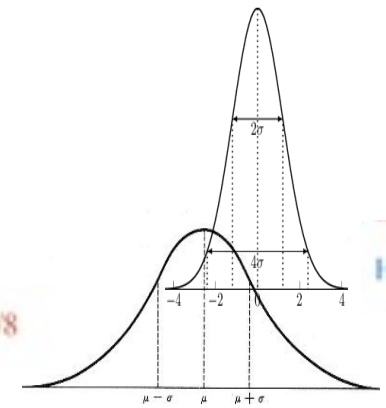
- Since half the frequencies of the signal have been removed at each step, half of the samples can be discarded according to Nyquist's rule. At each step the filter outputs are therefore downsampled by 2

- At each step:
  - Halves the time resolution (only half number of samples resulted)
  - Doubles the frequency resolution (the spanned frequency band halved)



A continuous version example





High-pass filter h_1: (-1 2 -1)/2

Low-pass filter h_0: (-1 2 6 2 -1)/8

1D signal ...100 100 100 100 200 200 200 200...

-100+200+600+400-200= 900/8 = 112,5

Before downsampling:

...	100	100	87.5	112.5	187.5	212.5	200	200...
...	0	0	0	-50	50	0	0	0...

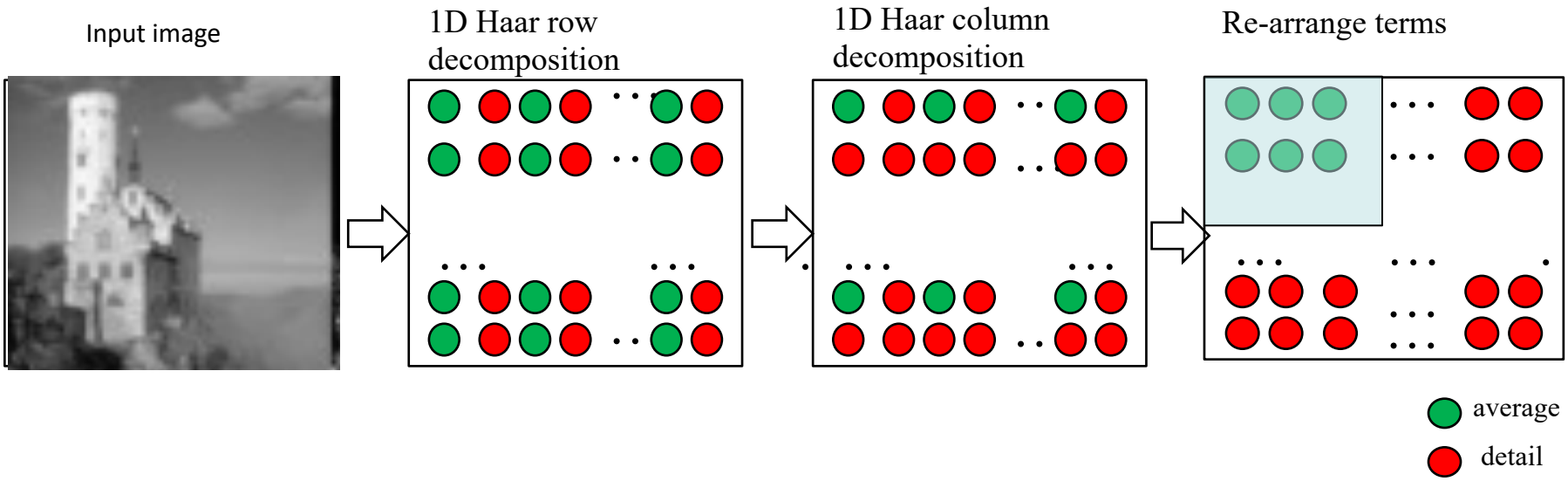
-100+400-200 = 100/2 = 50

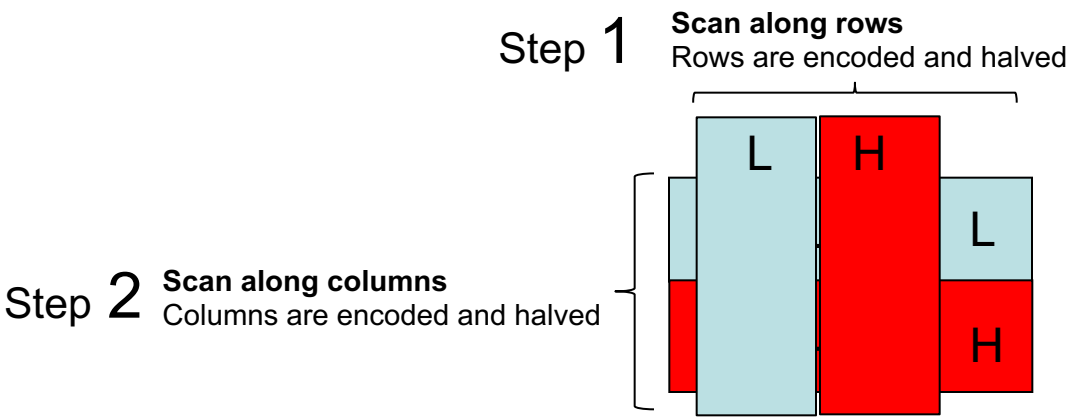
After downsampling:

... 100	112.5	212.5	200...	
... 0	0	50	0	...

2D Discrete Wavelet Transform

- In the two-dimensional case, the *Mallat method* is applied
  - Compute 1D Haar wavelet decomposition of each row of the original pixel values.
  - Compute 1D Haar wavelet decomposition of each column of the row-transformed pixels.



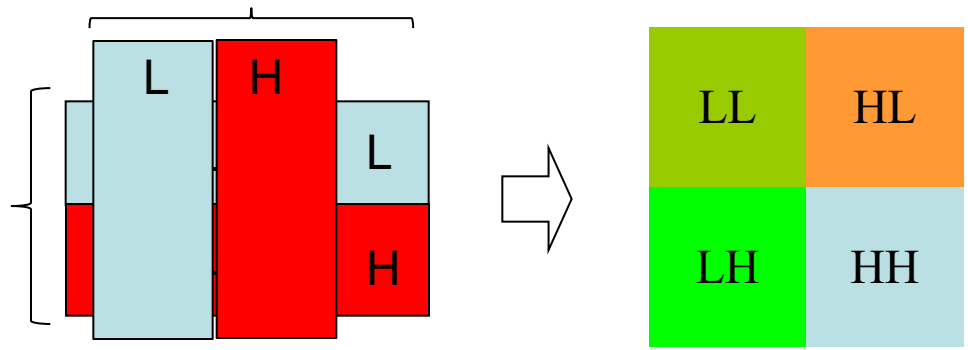


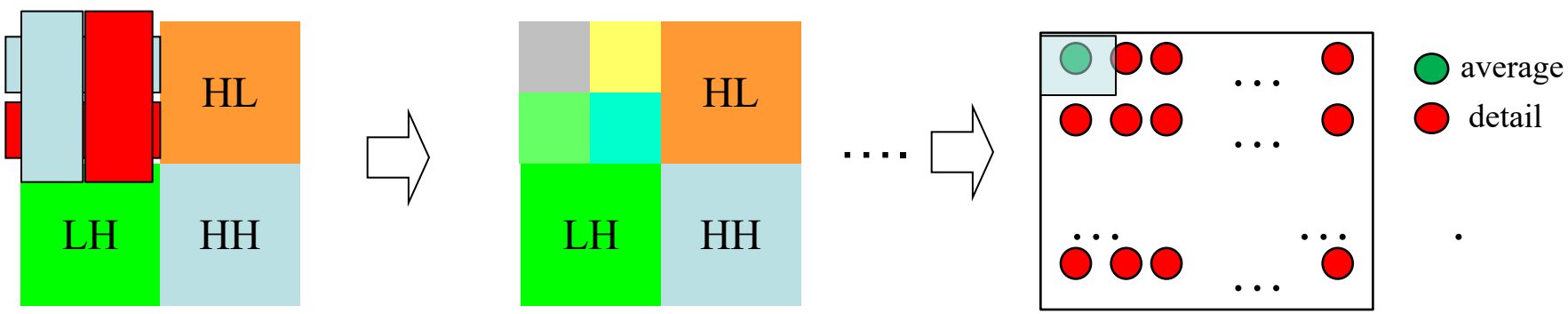
2D Discrete Wavelet Transform

- Decomposition is iteratively applied: the LL band is recursively decomposed, first horizontally and then vertically.
- Since downsampling is performed at each pass, at each iteration the image halves its size in the horizontal and vertical directions.

Step 1 **Scan along rows**
Rows are encoded and halved

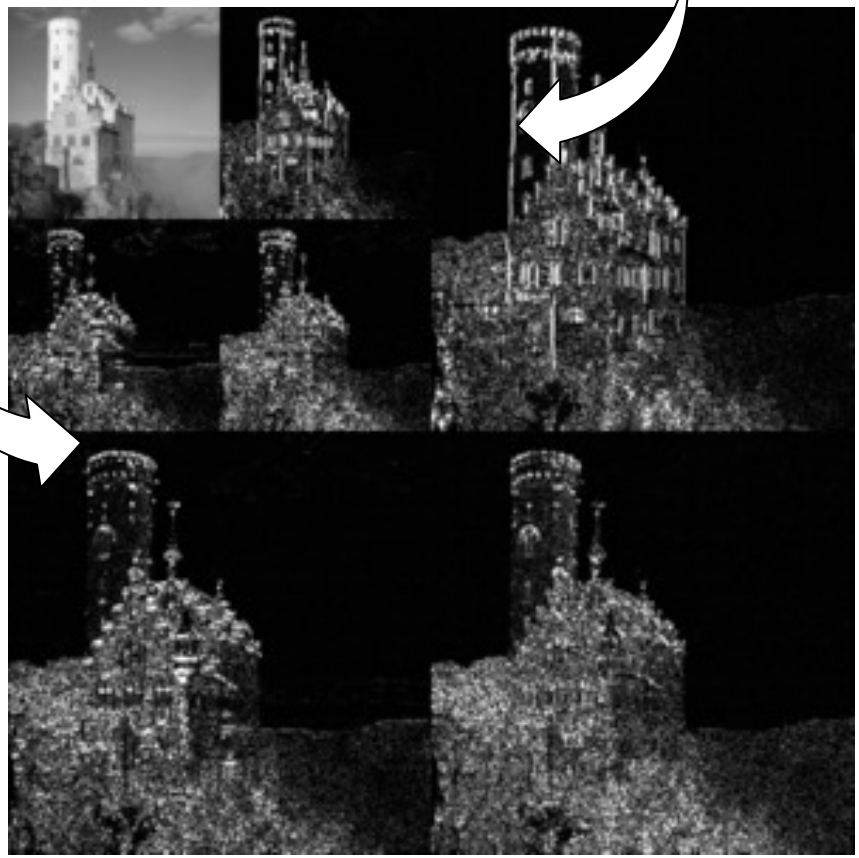
Step 2 **Scan along columns**
Columns are encoded and halved





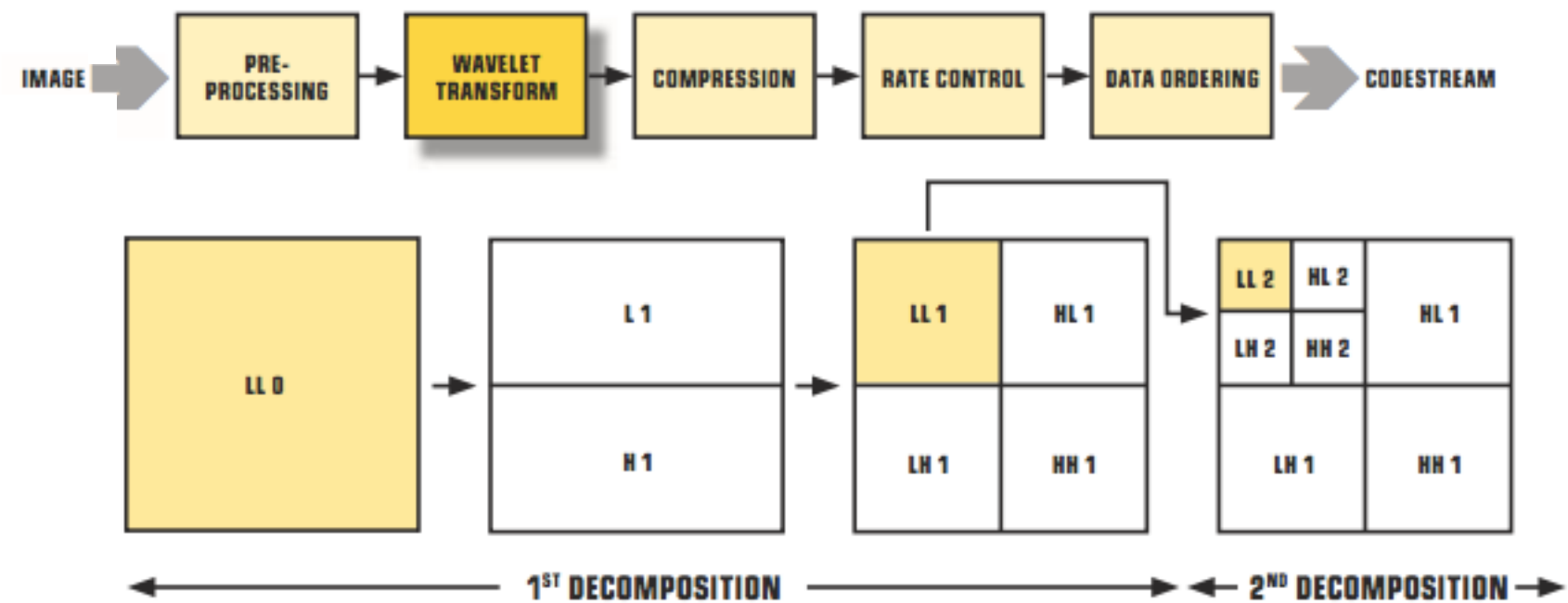
2D Wavelet Decomposition

- Conceptually, for a particular image, these subbands translate to:
  - low-frequency approximation of the original (LL)
  - primarily vertical edges (HL)
  - primarily horizontal edges (LH)
  - diagonal edges (HH).



An example

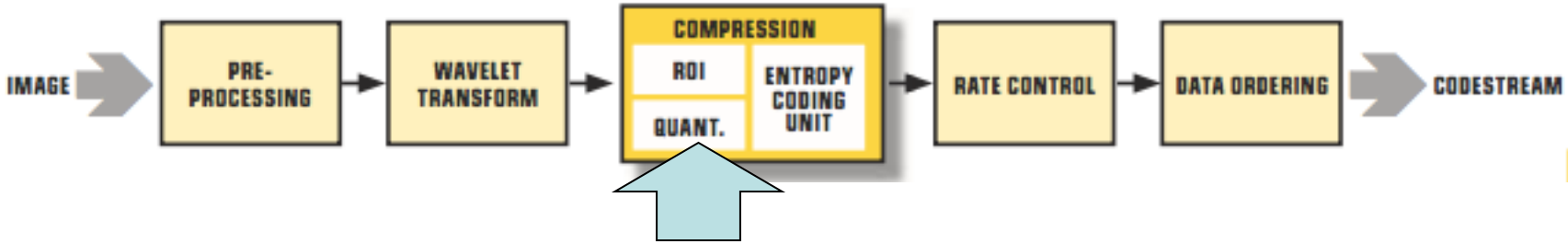
- By itself the Wavelet transform does not perform any compression of image data. It only reorganizes image content in a way that is easier to compress.
- Recursively passing the image through a low pass and high pass filter performs decorrelation of information and concentrates the image information in a small, localized area.
- The result is a collection of *sub-bands* which represent several approximation scales i.e. a multi-resolution image representation. A sub-band is a set of coefficients, real numbers which represent aspects of the image associated with a certain frequency range as well as a spatial area of the image.



JPEG2000 quantization

- After Discrete Wavelet decompostion, quantization matrices can be applied to the decomposed image to serve a diverse set of applications. JPEG2000 does not specify the use of particular quantization matrices.

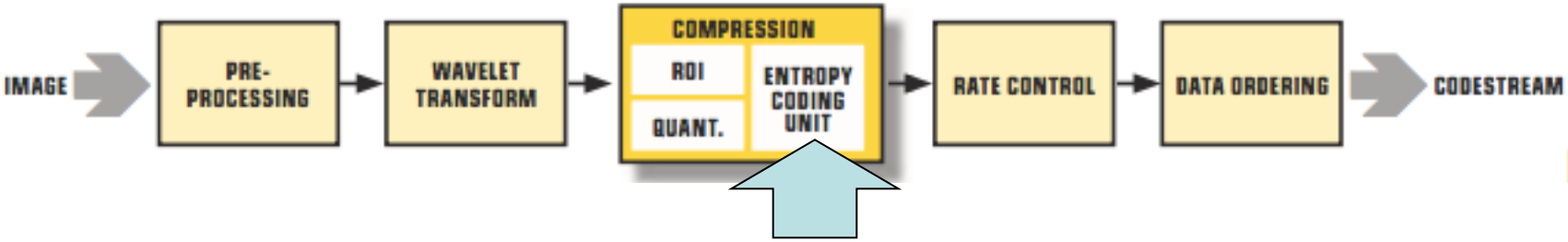
- Uniform quantization is performed within each subband, with different levels of quantization for each subband. Generally, the higher frequency subbands are quantized more coarsely, since humans have lower contrast sensitivity to high frequency information.



JPEG2000 entropy encoding

- The quantized sub-bands are split into rectangular regions in the wavelet domain and split further into *code blocks*.

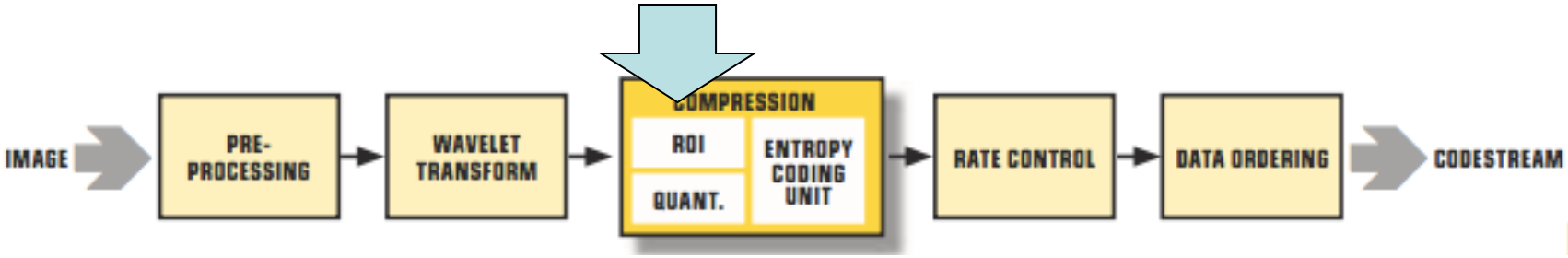
- They are encoded with *binary MQ-coder* (entropy encoding). Binary MQ-coder is a better approximation of entropy when the probabilities of the various symbols are not fixed.
  In many cases, the probability varies, so the average number of bits per symbol in a Huffman coding scheme could be much larger than the entropy.



Region of Interest (ROI) coding

- Thanks to components, tiling and multiresolution coding, JPEG 2000 offers increased flexibility with respect to JPEG, allowing ROI coding

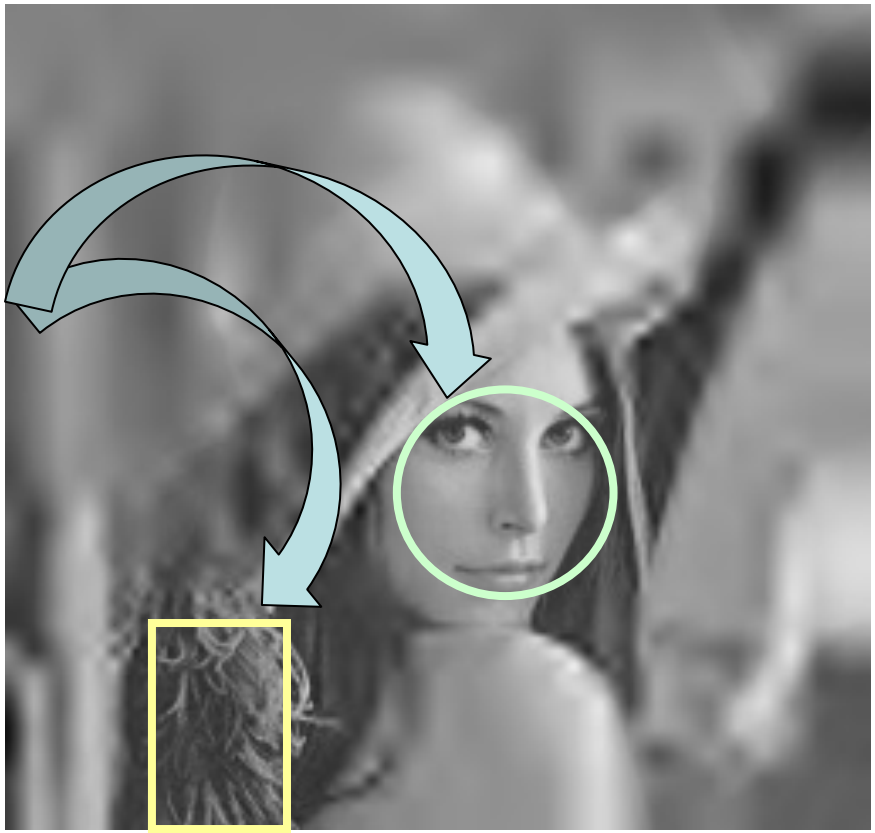
- In ROI coding, portions of an image are stored at higher quality than the rest of the image. This is useful, because we may care more about detail in some portions of an image than in others.



- ROI is easy to do when the image is stored compressed in a multi resolution format.

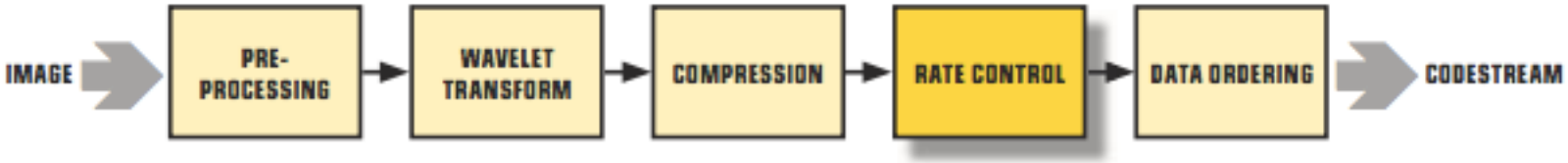
- Start with a ROI mask, which marks out a region of the image to store at higher quality. The wavelet coefficients corresponding to the transform of the mask have to be stored at higher quality (quantized less coarsely). Apply the transform and look at which coefficients fall in the mask.

Coefficients here are quantized less coarsely at any subband



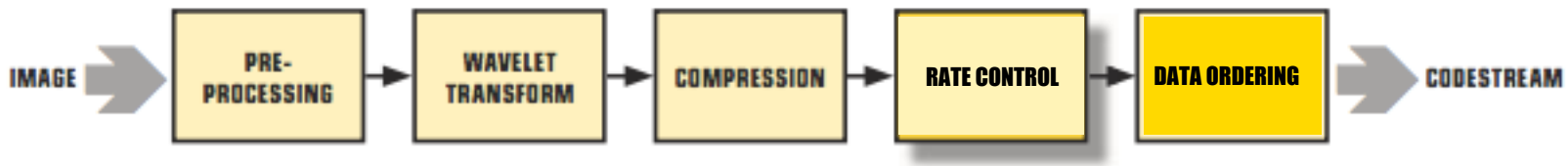
JPEG2000 rate control

- The simplest possibility for bitrate control is discarding resolutions: only several of the smallest Wavelet transform resolution levels get transmitted to the client in such a case
- This option provides a fast way of progressive quality improvement with no special demands/settings during image encoding

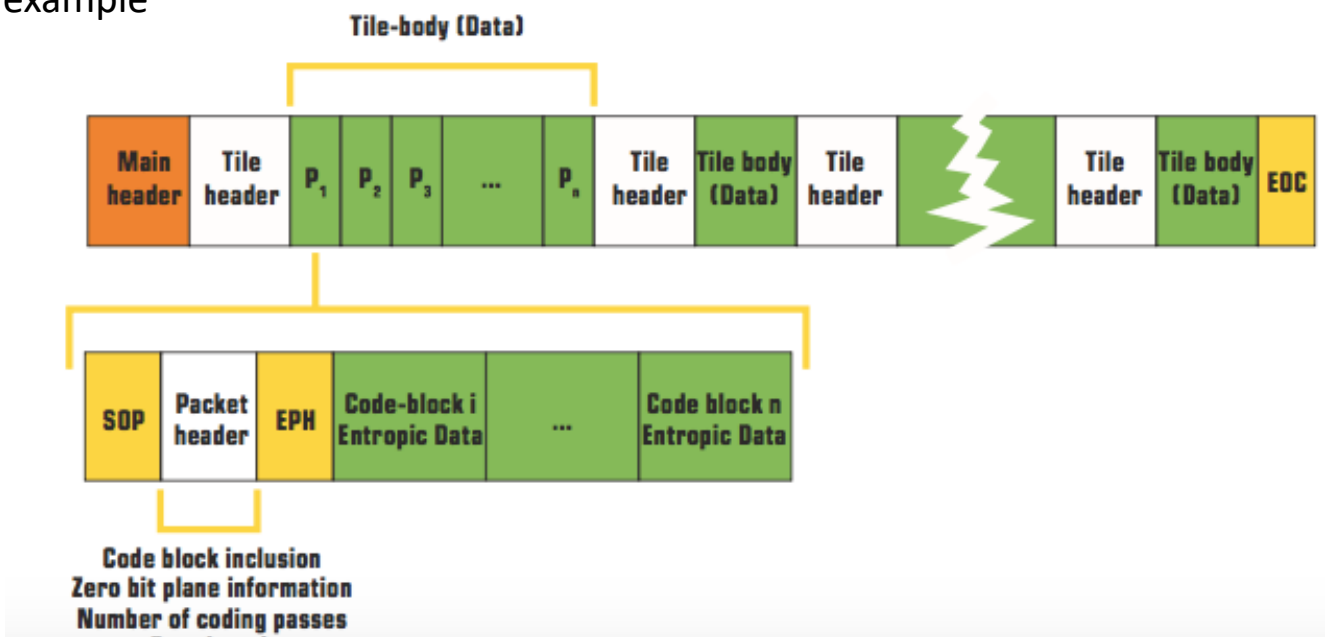


JPEG2000 data ordering

- The data ordering unit creates a succesion of packets (the JPEG2000 codestream)



JPEG2000 codestream example



JPEG2000 vs JPEG

- With respect to JPEG it allows space saving in the order of 20%-30%. Therefore it appears to be particularly suited for large images.

- The main advantage offered by JPEG 2000 is the flexibility of the codestream which is scalable in nature, meaning that it can be decoded in a number of ways. By truncating the codestream at any point one may obtain a representation of the image at a lower resolution, or signal-to-noise ratio .

- In this way JPEG 2000 can arrange a large range of bit rates (both very low and very high compression rates are supported). With JPEG if we want to trasmit over low bit rate we should first reduce the resolution and then encode.

A comparative example

- The wavelet basis functions are smoother than the DCT basis functions which tend to be blocky and are more natural and pleasing to the eye. JPEG 2000 only produces ringing artifacts, manifested as blur and rings near edges in the image

JPEG at 0.125 bpp (enlarged)

C. Christopoulos, A. Skodras, T. Ebrahimi, JPEG2000 (online tutorial)

JPEG2000 at 0.125 bpp

C. Christopoulos, A. Skodras, T. Ebrahimi, JPEG2000 (online tutorial)

JPEG2000 drawbacks

- As a consequence of its flexibility, JPEG 2000 requires encoders/decoders that are complex and computationally demanding.

- As of 2018, there are very few digital cameras that encode photos in the JPEG 2000 format.

- There are also few software and browsers for processing and visualizing
  - Adobe Photoshop has an Adobe plugin to manage JPEG2000 images
  - QickTime can read JPEG2000 images
  - Safari supports JPEG2000